

多变异策略的差分演化算法研究

黄 华¹, 刘 罡², 李会珍³

(1. 武汉学院信息工程学院, 湖北 武汉 430212;

2. 湖北工业大学计算机学院, 湖北 武汉 430068;

3. 武汉纺织大学数理科学学院, 湖北 武汉 430200)

摘要: 针对经典差分演化算法易出现全局搜索能力低、收敛速度慢的特点, 提出了一种多变异策略的差分演化算法。首先, 该算法在经典差分演化算法的基础上, 引入混沌映射机制进行初始化, 可以增强算法的全局搜索能力。然后, 选择多种变异策略进行优化: (1) 利用 Hilbert 变异矩阵有选择方向的进行变异, 减少变异的随机性; (2) 利用正交表来选择种群变异的分量, 增强全局搜索能力; (3) 利用 K-means 聚类算法, 对种群进行聚类选择, 从而加快算法的收敛速度。对多个经典测试函数进行试验研究, 研究结果表明: 该方法具有快速的收敛能力、良好的稳定性, 其优化性能显著提升。

关键词: 差分演化算法; 混沌映射; 正交设计; Hilbert 变异矩阵; K-means 聚类

中图分类号: TP18

文献标识码: A

文章编号: 2096-9759(2023)04-0051-04

Research on differential evolution algorithm with multiple mutation strategies

HUANG Hua¹, LIU Gang², LI Huizhen³

(1. School of Information Engineering of Wuhan College, Wuhan 430212, China;

2. Computer School of Hubei University of Technology, Wuhan 430086, China;

3. School of Mathematical and Physical Sciences of Wuhan Textile University, Wuhan 430200, China)

Abstract: Aiming at the characteristics of the classical differential evolution algorithm, such as low global search ability and slow convergence speed, a differential evolution algorithm with multiple mutation strategies is proposed. First, based on the standard differential evolution algorithm, the algorithm introduces chaotic mapping mechanism, which can enhance the global search ability of the algorithm; Then, we select a variety of mutation strategies to optimization, (1) use the Hilbert mutation matrix to select the direction of mutation to reduce the randomness of mutation, (2) use the orthogonal table to select the components of population mutation and enhance the global search ability, (3) use the K-means clustering algorithm to cluster the population, so as to speed up the convergence speed of the algorithm. The experimental results of several classical test functions show that the method has fast convergence ability, good stability, and its optimization performance is significantly improved.

Key words: Differential evolution algorithm; Chaotic mapping; Orthogonal design; Hilbert variation matrix; K-means clustering

1 引言

Storn 和 Price 于 1995 年提出的差分演化算法 (Differential Evolution, DE) 是一种简单有效的随机启发式优化算法^[1], 该算法对实数进行编码, 其原理简单、参数少、收敛快、鲁棒性强, 且全局优化能力相对其他算法较好, 优化效果好。但是, 和其他演化算法一样, 对于复杂问题 DE 算法也具有容易陷入局部最优的缺点, 且算法的优化结果对参数的选择比较敏感^[2]。为了提高 DE 算法的优化效果, 很多学者针对了选择最优的参数, 对算法中的变异策略不断的改进^[3-5]。这些改进变异策略提高了算法的优化性能, 但在处理较复杂高维问题时, 全局优化能力还存在一些缺陷, 收敛精度得不到明显改善。

自适应差分演化算法 (Improved Adaptive Differential Evolution, JADE) 虽然某种程度上提高了算法的优化性能, 但也不容易得到全局的最优解, 而且存在收敛精度低、速度慢的缺点。针对这些问题, Zhang 等人^[6]在 2009 年提出了带归档操作的自适应差分演化算法, 该算法通过对历史数据进行归档操作来提供一些信息, 可以提高算法的收敛效果, 并能以自动调整控

制参数的方式来提升算法的稳定性。Mohamed 等人^[7]改进了 JADE 算法的变异算子, 在 2018 年提出了一种增强的适应性微分变换差分进化算法, 该算法通过引入三角形变异算子, 来协调全局搜索能力与局部搜索能力之间的关系, 同时使用了两种自适应方案来控制参数选择, 在一定程度上提高了算法的全局搜索性能。

本文针对差分演化算法中存在的不足, 提出一种多变异策略的差分演化算法 (Differential Evolution with multiple Mutation Strategy, MS-DE)。首先, 在种群初始化时将混沌映射矩阵融合到 DE 算法中, 利用混沌映射矩阵的随机性、规律性及遍历性等特点来提升算法的全局搜索能力。其次, 我们基于 DE 算法引入正交设计^[8]、Hilbert 矩阵变换和 K-means 聚类算法, 来提升算法的收敛速度。最后, 我们对多种经典测试函数进行实验, 结果表明: 本文所提出的算法优于其他一些性能较好的遗传算法和优化算法, 所提算法不仅能有效避免早熟收敛, 还保证了全局收敛能力。

本文余下的部分安排如下。第二部分介绍经典的差分演

收稿日期: 2023-02-13

基金项目: 武汉学院校级科研项目 (X2022030); 湖北省教育厅科学技术研究计划指导性项目 (B2021365); 湖北省自然科学基金指导性项目 (2022CFC065); 湖北省高校优秀中青年科技创新团队 (T2022055)。

作者简介: 黄华 (1992-), 男, 湖北孝感人, 讲师, 主要研究方向: 深度学习; 刘罡 (1982-), 男, 湖北武汉人, 副教授, 主要研究方向: 深度学习; 李会珍 (1990-), 女, 湖北武汉人, 讲师, 主要研究方向: 机器学习与图像处理。

化算法,第三部分在经典的差分演化算法基础上提出算法的变异策略,第四部分将所提算法在经典测试函数上进行实验验证,并对实验结果进行评价和分析,第五部分对全文进行总结。

2 差分演化算法

差分演化算法是基于种群的自适应全局优化算法,主要通过差分变异算子、交叉算子和选择算子等对初始种群不断进行迭代演化,其演化过程类似于遗传算法。下面介绍经典DE算法的主要步骤。

步骤1 初始化种群: D 表示目标问题的自变量维数, NP 表示种群数量, X_i 表示种群中的第 i 个个体, 定义为: $X_i = (x_{i1}, x_{i2}, \dots, x_{iD})$ 。在规定的范围内随机生成 NP 个 D 维向量组成初始种群。

步骤2 差分变异算子: 从种群中任意挑选一个个体 X_i , 按公式(1)生成对应的变异个体 V_i :

$$V_i = X_{r_1} + \mu \times (X_{r_2} - X_{r_3}) \quad (1)$$

这里 X_{r_1} 、 X_{r_2} 和 X_{r_3} 为当前种群中随机挑选的 3 个异于 X_i 且互不相同个体; $\mu \in [0, 1]$ 表示缩放因子。

步骤3 交叉算子: 对个体 X_i 和变异个体 V_i 按公式(2)进行随机离散杂交, 得到新的测试个体 U_i :

$$u_{ij} = \begin{cases} v_{ij}, & \text{rand}(0,1) \leq \beta \text{ or } j = j_{\text{rand}} \\ x_{ij}, & \text{otherwise} \end{cases} \quad (2)$$

其中 $\text{rand}(0,1)$ 表示 [0,1] 内的均匀随机小数; j_{rand} 表示 [1, D] 内的均匀随机整数, 从而可以保证 U_i 中至少有一维数据进行了变异; $\beta \in [0, 1]$ 表示发生变异的概率。

步骤4 选择算子: 对交叉得到的个体 U_i , 采用如下的优化选择方法:

$$X'_i = \begin{cases} U_i, & f(U_i) \leq f(X_i) \\ X_i, & \text{otherwise} \end{cases} \quad (3)$$

其中 $f(X_i)$ 表示 X_i 的适应值; X'_i 表示代替 X_i 而进入下一代的子个体。

步骤5 判断是否终止算法: 通过差分变异算子、交叉算子和选择算子对种群的作用, 可以得到新的种群 $\{X'_i | i=1, 2, \dots, NP\}$ 进入下一代, 按照这种方式迭代直至达到终止条件, 从而终止算法输出最优结果。

3 差分演化算法的改进变异策略

3.1 混沌映射初始化

混沌映射矩阵采用 1 维 Logistics 映射的方式进行种群初始化, 迭代方程如下:

$$x_{k+1} = ux_k(1-x_k) \quad (4)$$

其中 x_k 表示混沌矩阵的变量, 初始变量 $x_1 \in [0, 1]$; u 表示控制参数, $u \in (1, 4]$, 当 u 取 4, 系统处于完全混沌状态, 此时 $x_1 \notin [0, 0.25, 0.5, 0.75, 1.00]$ 。种群初始化的改进策略主要是将均匀随机分布产生的初始种群与混沌映射产生的初始种群进行竞争, 选出较好的个体作为初始种群, 具体操作如下:

步骤1 均匀随机产生初始种群 $R(NP)$

$$x_{ij} = \text{rand} \times (x_{j\max} - x_{j\min}) \quad (5)$$

其中, $i \in [1, NP]$, $j \in [1, D]$, rand 表示 0~1 之间的一个均匀随机小数, $x_{j\max}$ 以及 $x_{j\min}$ 分别是第 j 个变量的上下界。

步骤2 生成 (0,1) 之间的一个随机数 $x_{1,1}$, 且 $x_{1,1}$ 不可取

0.25, 0.5, 0.75, 从 $x_{1,1}$ 开始, 按照公式(4)生成 $NP \times D$ 的混沌矩阵。

$$\begin{bmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,n} \\ x_{2,1} & x_{2,2} & \cdots & x_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{NP,1} & x_{NP,1} & \cdots & x_{NP,1} \end{bmatrix} \quad (6)$$

步骤3 将混沌矩阵中的元素 $x_{i,j}$ 映射到 $[x_{j\min}, x_{j\max}]$ 中, 从而得到混沌初始种群 $C(NP)$ 。

步骤4 计算种群 $R(NP)$ 和 $C(NP)$ 中所有个体的适应性值, 取最优的前 NP 个个体作为初始种群 P 。

3.2 Hilbert 变异矩阵

为加快算法的收敛速度, 引入如下的 Hilbert 矩阵:

$$H = (a(i, j))_{D \times D} \quad (7)$$

其中 $a(i, j) = 1/(i+j-1)$ 。令种群矩阵 $P_{NP \times D}$ 乘以 Hilbert 矩阵 $H_{D \times D}$, 得到一个新的种群, 再与原种群对应比较进行选择产生新的种群。

3.3 正交设计变异

(1) 正交表设计

这里采用 2 水平正交表 $L_2(R, C)$, 其中 $R=2^j$ 表示正交表的行数, $C=2^j-1$ 表示正交表的列数, 且设计的正交表满足如下关系:

$$\begin{aligned} \min J &= 2^j \\ \text{s.t. } C &= 2^j - 1 \geq D \end{aligned} \quad (8)$$

其中, D 表示种群变量的维数。在算法中找到满足公式(7)最小的 J 。选择合适的正交表之后, 采用文献[1]中的正交表产生方法生成正交表的基本列, 然后再生成非基本列来产生正交表。

(2) 正交子群

得到正交表 $L_2(R, C)$ 后, 随机选择两个个体 $X_1 = (x_{11}, x_{12}, \dots, x_{1D})$ 和 $X_2 = (x_{21}, x_{22}, \dots, x_{2D})$ 进行 2 水平正交杂交, 生成正交子群 $O(NP, D)$, 具体操作如下:

```
for i=1 to NP
  for j=1 to D
    k=L(I,j)
    if k==1 then
      O(i,j)=x1j
    else if
      O(i,j)=x2j
    end if
  end for
end for.
k-means 聚类
```

3.4 k-means 聚类

为了进一步加快收敛, 引入标准的 K-means 聚类算法, 计算出种群的 k 个聚类中心, 并把 k 个聚类中心作为个体, 与种群竞争, 留下最优的种群。

K-means 聚类算法的具体操作如下:

步骤1 随机选择初始种群的 k 个个体作为初始聚类中心。

$$a = a_1, a_2, \dots, a_k \quad (9)$$

步骤2 针对种群中的每个个体 X_i 计算它到 k 个聚类中

心的距离,并将其分配到距离最小的聚类中心所对应的类 c_i 中;

步骤 3 针对每个类别 a_j ,重新计算它的聚类中心。

$$a_j = \frac{1}{|c_j|} \sum_{x \in c_j} x \quad (10)$$

其中 $|c_i|$ 表示类 c_i 中个体的个数。

步骤 4 重复前面步骤 2、3 两步操作,直至达到终止条件。

4 实验结果

我们对 20 个经典测试函数(见表 3)求函数的最小值,分别在函数变量为 30 维和 100 维时进行实验,其中种群规模为 40,迭代 1000 次,并与经典差分演化算法 DE、布谷鸟搜索算法(CS)^[9]、花授粉算法(FPA)^[10]、改进的粒子群算法(PO-PSO)^[11]的结果进行比较,对比结果如下(最好的结果用黑体标出):

表 1 几类经典算法对于经典函数在 30 维上的统计结果

f	CS	DE	FPA	PO-PSO	MS-DE
f ₁	5.85E-10	1.21E+01	5.73E-02	3.34E-16	1.32E-53
f ₂	7.16E+00	3.09E+03	7.48E+01	2.79E-16	3.90E-44
f ₃	5.65E-04	8.09E-01	4.52E-03	4.15E-19	5.75E-51
f ₄	6.02E-19	6.02E-19	9.28E-17	1.59E-62	1.75E-94
f ₅	5.52E-01	3.30E+00	3.27E-01	3.45E-08	3.62E-48
f ₆	3.23E+00	1.39E+01	5.17E+00	1.77E-02	5.35E-01
f ₇	0.00E+00	1.25E+01	0.00E+00	0.00E+00	0.00E+00
f ₈	7.21E-66	3.51E-49	7.19E-66	7.17E-66	0.00E+00
f ₉	3.92E-02	8.86E-02	3.37E-02	3.16E-02	3.17E-02
f ₁₀	3.59E+01	8.48E+01	3.29E+01	6.69E+01	2.75E+01
f ₁₁	7.25E+01	1.82E+02	1.49E+02	3.44E+01	1.39E+01
f ₁₂	5.01E-02	1.05E+00	2.42E-01	2.68E-14	2.22E-02
f ₁₃	3.96E+03	7.75E+03	6.79E+03	4.01E+03	8.07E-02
f ₁₄	1.96E+01	3.89E+00	2.00E+01	6.64E-11	3.15E-14
f ₁₅	5.26E-01	1.97E+00	1.58E-02	9.46E-20	1.28E+05
f ₁₆	3.19E-01	3.07E+00	2.19E-02	8.59E-16	4.10E+02
f ₁₇	6.43E+00	1.20E+01	7.36E+00	3.60E-05	3.86E-02
f ₁₈	2.58E+03	2.58E+03	2.58E+03	2.58E+03	3.66E-01
f ₁₉	7.50E+00	5.61E+00	2.75E+00	2.43E+00	1.08E+01
f ₂₀	5.91E+01	1.83E+02	1.22E+02	1.99E+01	0.00E+00

表 2 几类经典算法对于经典函数在 100 维上的统计结果

f	CS	DE	FPA	PO-PSO	MS-DE
f ₁	4.35E+06	6.33E+03	7.38E+02	6.65E-03	5.79E-19
f ₂	4.35E+06	1.34E+07	1.77E+06	1.01E+03	9.02E-18
f ₃	9.11E+01	2.36E+03	2.72E+02	3.38E-02	2.09E-25
f ₄	1.00E-16	2.93E-05	1.22E-11	4.14E-21	2.56E-49
f ₅	1.00E+10	7.30E+01	2.52E+01	7.81E-01	4.87E-33
f ₆	1.60E+01	7.94E+01	2.27E+01	2.81E+00	3.93E+01
f ₇	4.89E+02	7.00E+03	7.01E+02	9.50E+00	0.00E+00
f ₈	2.97E-205	1.57E-81	5.10E-202	7.12E-218	0.00E+00
f ₉	8.41E-01	3.89E+00	6.72E-01	1.21E-01	5.82E-02
f ₁₀	4.34E+03	4.90E+04	5.99E+03	2.28E+02	9.89E+01

f	CS	DE	FPA	PO-PSO	MS-DE
f ₁₁	4.49E+02	9.65E+02	7.93E+02	1.04E+02	1.99E+00
f ₁₂	2.86E+00	5.50E+01	6.85E+00	4.43E-03	9.90E-03
f ₁₃	1.97E+04	3.30E+04	3.09E+04	1.60E+04	3.71E-01
f ₁₄	1.99E+01	1.45E+01	2.00E+01	1.58E+00	3.90E-09
f ₁₅	1.00E+10	6.15E+07	4.64E+04	6.09E-02	6.66E+06
f ₁₆	1.00E+10	8.57E+07	4.51E+04	1.64E-02	1.81E+06
f ₁₇	5.03E+01	6.85E+01	4.75E+01	2.19E-01	1.16E-01
f ₁₈	8.88E+03	8.90E+03	8.84E+03	8.79E+03	2.00E+01
f ₁₉	5.18E+01	6.40E+01	4.44E+01	1.91E+01	1.55E+00
f ₂₀	4.30E+02	8.48E+02	6.88E+02	9.84E+01	1.17E-12

表 3 经典测试函数集

Name	Function	Range	min
Sphere	$f_1(x) = \sum_{i=1}^D x_i^2$	$[-100, 100]^D$	0
Elliptic	$f_2(x) = \sum_{i=1}^D (10^{65})^{i-1} x_i^2$	$[-100, 100]^D$	0
SumSquare	$f_3(x) = \sum_{i=1}^D ix_i^2$	$[-10, 10]^D$	0
SumPower	$f_4(x) = \sum_{i=1}^D x_i ^{2.1}$	$[-1, 1]^D$	0
Schwefel2.22	$f_5(x) = \sum_{i=1}^D x_i + \prod_{i=1}^D x_i $	$[-10, 10]^D$	0
Schwefel2.21	$f_6(x) = \max\{ x_i , 1 \leq i \leq D\}$	$[-100, 100]^D$	0
Step	$f_7(x) = \sum_{i=1}^D [0.5 + x_i]^2$	$[-100, 100]^D$	0
Exponential	$f_8(x) = \exp(-0.5 \sum_{i=1}^D x_i^2)$	$[-10, 10]^D$	0
Quartic	$f_9(x) = \sum_{i=1}^D ix_i^4 + \text{rand}[0, 1]$	$[-1.3, 1.3]^D$	0
Rosenbrock	$f_{10}(x) = \sum_{i=1}^{D-1} [100(x_{i+1} - x_i^2)^2 + (x_i - 1)^2]$	$[-10, 10]^D$	0
Rastrigin	$f_{11}(x) = \sum_{i=1}^D (x_i^2 - 10 \cos(2\pi x_i) + 10)$	$[-5, 5]^D$	0
Griewank	$f_{12}(x) = \frac{1}{4000} \sum_{i=1}^D x_i^2 - \prod_{i=1}^D \cos \frac{x_i}{\sqrt{i}} + 1$	$[-600, 600]^D$	0
Schwefel2.26	$f_{13}(x) = 418.98288 - \sum_{i=1}^D x_i \sin \sqrt{ x_i }$	$[-500, 500]^D$	0
Ackley	$f_{14}(x) = 20 + e - 20 \exp(-0.2 \sqrt{\sum_{i=1}^D x_i^2 / D}) - \exp(\sum_{i=1}^D \cos(2\pi x_i) / D)$	$[-50, 50]^D$	0
Penalized1	$f_{15}(x) = \frac{\pi}{D} (10 \sin^2(\pi \frac{x_1+5}{4}) + \sum_{i=1}^{D-1} (\frac{x_i+1}{4})^2 (1 + 10 \sin^2(\pi \frac{x_{i+1}+1}{4}) + (\sum_{j=i+1}^D x_j^2)^2) + \sum_{i=1}^D u(x_i, 10, 100, 4)$ $u_{x_i, \mu, \sigma, \mu} = \begin{cases} k(x_i - \mu)^{\mu}, & x_i > \mu \\ 0, & -\sigma \leq x_i \leq \mu \\ k(-x_i - \mu)^{\mu}, & x_i < -\sigma \end{cases}$	$[-100, 100]^D$	0
Penalized2	$f_{16}(x) = \frac{1}{10} \{ \sin^2(\pi x_1) + \sum_{i=1}^D (x_i - 1)^2 [1 + \sin^2(3\pi x_{i+1})] + (x_D - 1)^2 \} + \sum_{i=1}^D u(x_i, 5, 100, 4)$	$[-100, 100]^D$	0
Alpine	$f_{17}(x) = \sum_{i=1}^D x_i \sin x_i + 0.1 x_i $	$[-10, 10]^D$	0
Lévy	$f_{18}(x) = \sum_{i=1}^D (x_i - 1)^2 (1 + (\sin(3\pi x_{i+1}))^2) + x_D - 1 (1 + (\sin(3\pi x_D))^2)$	$[-10, 10]^D$	0
Weierstrass	$f_{19}(x) = \sum_{i=0}^D \sum_{k=0}^{20} 0.5^k \cos(2\pi 3^k (x_i + 0.5)) - D \sum_{k=0}^{20} 0.5^k \cos(3^k \pi)$	$[-0.5, 0.5]^D$	0
Mis-scaleRastrigin 100	$f_{20}(x) = \sum_{i=1}^D ((100^{(i-1)/(D-1)} x_i)^2 - 10 \cos(2\pi 100^{(i-1)/(D-1)} x_i) + 10)$	$[-0.5, 0.5]^D$	0

5 实验结果分析

本文针对差分演化算法易出现全局搜索能力低、收敛速度慢的特点,提出一种多变异策略的快速差分演化算法。首先该算法在标准差分演化算法的基础上,引入混沌映射机制,

基于计算机视觉的灰度图像特征点快速匹配方法

刘玉洁, 张 丽

(郑州科技学院 信息工程学院, 河南 郑州 450064)

摘要: 常规的图像特征点匹配方法多数采用光流跟踪法, 具有一定的局限性且特征点匹配率较低, 无法实现短时间内对灰度图像特征点精确匹配的目标。针对这一问题, 文章引入计算机视觉技术, 提出了一种全新的灰度图像特征点快速匹配方法。首先, 依据高斯函数理念, 得出特征点相似程度。其次, 融合处理灰度图像, 得出特征点灰度值。在此基础上, 引入计算机视觉技术, 实现多维度地匹配灰度图像特征点。实验分析结果可知, 通过提出方法匹配灰度图像特征点, 在短期内, 特征点匹配正确的点数较多, 匹配率均在 99% 以上, 匹配效果优势显著。

关键词: 计算机视觉; 灰度图像; 特征点匹配; 高斯函数

中图分类号: TM933

文献标识码: A

文章编号: 2096-9759(2023)04-0054-03

0 引言

灰度图像特征点均具有鲜明的特性, 作为灰度图像中的重要组成部分^[1-4], 对实现灰度图像^[5]匹配操作具有直接影响, 因此, 科学合理的灰度图像特征点快速匹配方法至关重要。文献[6]林伟文等人提出的视觉同步定位与建图中特征点匹配算法优化, 采用 KITTI 数据集分别对 ORB-SLAM2 与基于 ORB-SLAM2 对特征点匹配算法优化进行实验验证。文献[7]张峻琦等人提出融合 RANSAC 光流跟踪法和特征点匹配法, 大大降低了光流跟踪法存在的误匹配。但是上述两种方法无法在短时间内对特征点作出精确的匹配, 导致特征点匹配率较低。

为了改善上述问题, 本文引入计算机视觉技术, 提出了一种全新的灰度图像特征点快速匹配方法, 为促进遥感图像与医学图像领域的研究发展提供保障。

1 灰度图像特征点快速匹配方法设计

1.1 灰度图像特征检测

在本文设计的灰度图像特征点快速匹配方法中, 首先, 依据高斯函数的相关理念, 对灰度图像特征进行全面检测, 通过建立灰度图像多尺度空间, 检测各个尺度下, 灰度图像存在的特征点。

已知一个二维灰度图像 $I(x, y)$, 依据高斯函数与图像卷积运算的方法, 构建二维灰度图像的尺度空间 $L(x, y, s)$, 其表达式为:

$$L(x, y, \sigma) = G(x, y, \sigma) \otimes I(x, y) \quad (1)$$

$$G(x, y, \sigma) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) \quad (2)$$

收稿日期: 2023-01-05

作者简介: 刘玉洁 (1994-), 女, 河南郑州人, 硕士研究生, 助教, 主要研究方向为计算机视觉、图像处理。

对种群初始化, 可以增强算法的全局搜索能力; 其次多种变异策略同时使用, 先利用 Hilbert 变异矩阵进行有选择方向的变异, 减少变异的随机性, 接着用正交表来选择种群变异的分量, 增强全局搜索能力, 最后用 K-means 聚类算法, 对种群进行聚类选择, 来加快算法的收敛速度。此外, 通过对 20 个经典测试函数进行试验, 在 30 维和 100 维的情况下, 将最终结果与差分演化算法 (DE)、布谷鸟搜索算法 (CS)、花授粉算法 (FPA)、改进的粒子群算法 (PO-PSO) 的结果进行比较。实验结果表明, 本文方法对不同的测试函数均有较好的最优结果, 验证了本文算法的有效性。

参考文献:

- [1] Storn R, Price K. Differential evolution - A Simple and efficient adaptive scheme for global optimization over Continuous Space[R]. University of California, Berkley: ICSI, 1995.
- [2] Storn R, Price K. Differential Evolution - A Simple and Efficient Heuristic for Global Optimization over Continuous Spaces [J]. Journal of Global Optimization (S0925-5001), 1997, 11(4): 341-359.
- [3] Brest J, Greiner S, Boskovic B. Self-adaptive control parameters in differential evolution: A comparative study on numerical benchmark problems[J]. IEEE Transactions on Evolutionary Computation, 2006, 10(6): 646-657.
- [4] Qin A K, Huang V L, Suganthan P N. Differential evolution algorithm with strategy adaptation for global numerical optimization[J]. IEEE Transactions on Evolutionary Computation, 2009, 13(2): 398-417.
- [5] 范泽华, 白铁成. 基于改进差分演化的高效图像分割算法. 现代电子技术, 2016, 39(14): 87-91.
- [6] Zhang J, Sanderson A C. JADE: Adaptive differential evolution with optional external archive[J]. IEEE Transactions on Evolutionary Computation, 2009, 13(5): 945-958.
- [7] Mohamed A W and Suganthan P N. Real-parameter unconstrained optimization based on enhanced fitness adaptive differential evolution algorithm with novel mutation[J]. Soft Computing, 2018, 22(10): 3215-3235.
- [8] 陈文霞, 龚文引, 蔡之华. 正交差分演化算法在工程优化设计中的应用. 计算机工程与应用, 2008, 44(18): 230-235.
- [9] Gandomi A, Yang X, Alavi A. Cuckoo search algorithm: a metaheuristic approach to solve structural optimization problems[J]. Engineering with Computers, 2013, 29(29): 17-35.
- [10] Yang X. Flower Pollination Algorithm for Global Optimization [M]. Unconventional Computation and Natural Computation, 2012: 240-249.
- [11] 曾彦翔. 一类改进的粒子群算法及其在分数阶系统参数估计中的应用[D]. 北京交通大学, 2020.